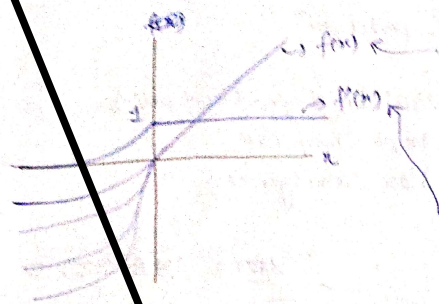


ELU - Exponential Linear Unit



always continuous & always differentiable

$$ELU(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

constant

$$ELU'(x) = \begin{cases} 1 & \text{if } x > 0 \\ ELU(x) + \alpha & \text{if } x \leq 0 \end{cases}$$

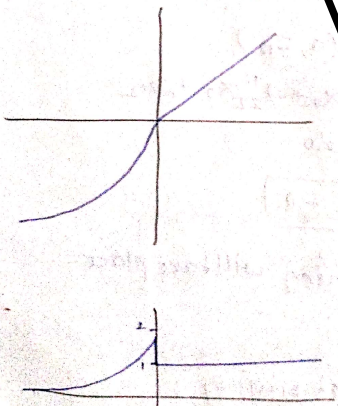
Advantages

- 1) close to zero centered
- 2) convergence faster
- 3) better generalized
- 4) dying relu
- 5) Always continuous & differentiable.

Disadvantages

computationally expensive.
(ex)

SELU - Scaled Exponential Linear Unit



$$SELU(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ e^x - x & \text{if } x \leq 0 \end{cases}$$

$\alpha \approx 1.6733$
 $\lambda \approx 1.0507$ } fixed parameters

$$SELU'(x) = \lambda \begin{cases} 1 & \text{if } x > 0 \\ e^x & \text{if } x \leq 0 \end{cases}$$

Extra Advantage

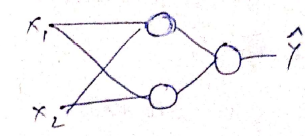
Self normalizing → activation
↓
normalized
↓
convergence faster
μ=0, std=1

Disadvantage

Research ongoing.

Weight Initialization Techniques

Why weight initialization is important?



- vanishing gradient problem
- exploding GP.
- slow convergence.

Wrong wt. init.

What to do?

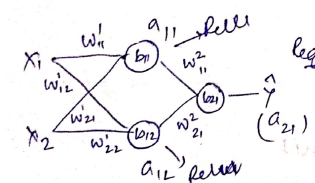
Sigmoid Activation

2000's Deep learning era

Wrong weight init.

Wrong techniques:

Case 1: Zero Initialization



Regression Explanation:

Relu, tanh, Sigmoid

$$a_{11} = \max(0, z_{11})$$

$$z_{11} = w'_{11}x_1 + w'_{21}x_2 + b_{11}$$

$$a_{12} = \max(0, z_{12})$$

$$z_{12} = w'_{12}x_1 + w'_{22}x_2 + b_{12}$$

$$a_{11} = 0, a_{12} = 0$$

$$a_{11} = a_{12} = 0$$

$$w = 0, b = 0$$

$$w'_{11} = w_{11} - \eta \left[\frac{\partial L}{\partial w_{11}} \right] = 0$$

no update. No training will take place.

Tanh

$$a_{11} = \frac{e^{z_{11}} - e^{-z_{11}}}{e^{z_{11}} + e^{-z_{11}}} = 0 \rightarrow \text{no update} \rightarrow \text{No training}$$

Sigmoid

$$a_{11} = \sigma(z_{11}) = 0.5$$

$$z_{11} = 0$$

$$a_{12} = \sigma(z_{12}) = 0.5$$

$$z_{12} = 0$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

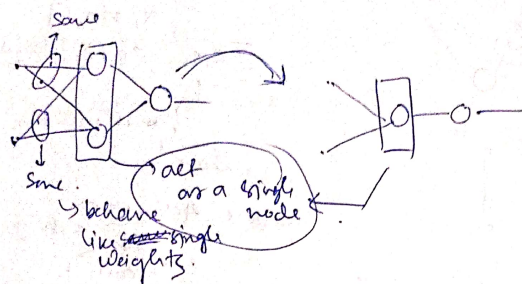
$$w = w - \eta \left[\frac{\partial L}{\partial w} \right]$$

$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} \frac{\partial z_{11}}{\partial w'_{11}}$$

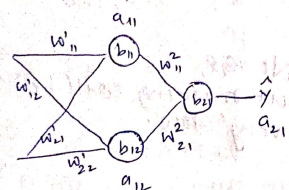
$$\frac{\partial L}{\partial w'_{12}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{12}} \frac{\partial z_{12}}{\partial w'_{12}}$$

$$\frac{\partial L}{\partial w'_{21}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{11}} \frac{\partial z_{11}}{\partial w'_{21}}$$

$$\frac{\partial L}{\partial w'_{22}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} \frac{\partial z_{12}}{\partial w'_{22}}$$



Case 2: Non-zero constant value.



$$w = 0.5, b = 0.5$$

$$\text{sigmoid} \approx 0$$

$$a_{11} = \max(0, z_{11}) \rightarrow \text{non zero}$$

$$z_{11} = w'_{11}x_1 + w'_{21}x_2 + b_{11} \neq 0$$

$$a_{12} = \max(0, z_{12}) \neq 0$$

$$z_{12} = w'_{12}x_1 + w'_{22}x_2 + b_{12} \neq 0$$

$$\text{here, } w'_{11} = w'_{22} = 0.5, w'_{21} = w'_{12} = 0.5, b_{11} = b_{12} = 0.5$$

$$\therefore z_{11} = z_{12} \therefore a_{11} = a_{12}$$

Activation Same

updates will be same.

So, they all nodes acts as a single neuron/node.

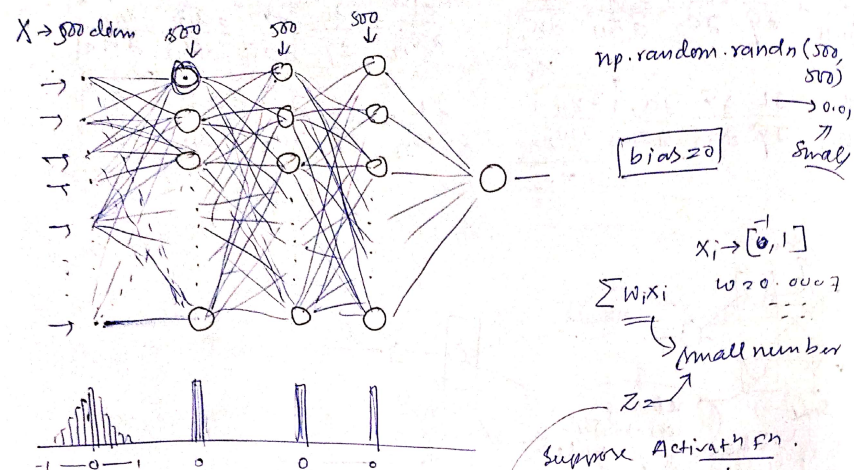
Case 3: Random Initialization

Small random values $\rightarrow$ tanh/sigmoid
Large random values.

data shape $\rightarrow (1000, 500)$

$x_1, \dots, x_{500}$

$\rightarrow$ gaussian distribution



$\text{np.random.randn}(500, 500)$
 $\rightarrow 0.0, \dots$
 $\rightarrow$ small

$x_i \rightarrow [0, 1]$
 $\sum w_i x_i$
 $\rightarrow$ small number

z_2

suppress Activation fn.

tanh

very close to zero.

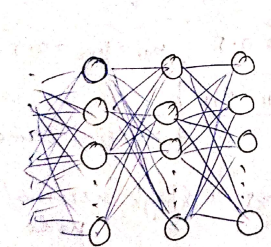
cannot update VGP.

$$\therefore \nabla \approx 0$$

$$w \approx w - \eta \nabla$$

in case of sigmoid $\rightarrow$ VGP, Relu $\rightarrow$ No VGP, but training will be very slow.

Random Initialization - Large values.



$\text{np.random.randn}(100, 500)$

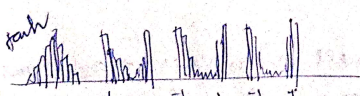
$[0, 1]$

$\sum w_i x_i \rightarrow 500$

$(0-1)$

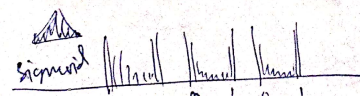
$(100 \rightarrow 500)$

tanh/sigmoid



Saturation

$\sum w_i x_i \rightarrow 1000 \rightarrow$ saturate



slow training

VGP.

Relu $\rightarrow$ if the inputs are large. or also very large $\rightarrow \nabla$ also very large

unstable gradient on can never converge

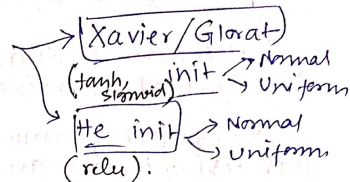
Key insights

Things not to do -

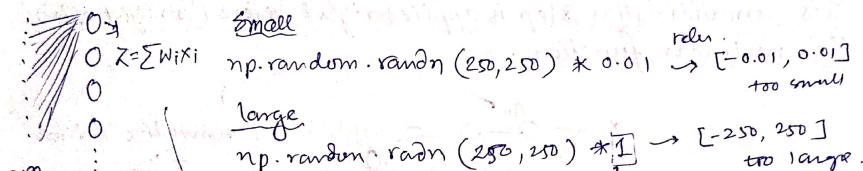
- ① Zero initialization. $\rightarrow$ no training
- ② Non-zero constant initialization. $\rightarrow$ linear
- ③ Random initialization with small weights. $\rightarrow$ slow convergence
- ④ Random initialization with large weights. $\rightarrow$ irregular convergence

What can be done?

$\rightarrow$ Heuristics (Jugad) $\rightarrow$ Practical Solution



Intuition



$$\text{Var} = \frac{1}{n} \rightarrow \cdot \sqrt{\frac{1}{250}} \rightarrow \text{std}$$

the goal is that,

- If the no. of inputs are large, then make sure that the z will not be too large.
- and if no. of inputs less, then make sure the value of z will not be less.

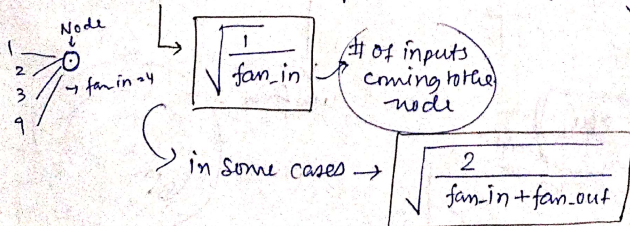
inputs of the previous layer

Suppose $n_{ip} = 250 \rightarrow$ large $\rightarrow \sqrt{250}$

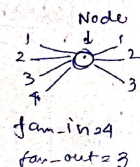
$n_{ip} = 5 \rightarrow$ small $\rightarrow \frac{1}{\sqrt{5}}$

Xavier Initialization. (applicable on tanh)

(normal) $\rightarrow$ np.random.randn(2, 2) * $\frac{1}{\sqrt{2}}$



He Normal (normal) $\rightarrow \sqrt{\frac{2}{\text{fan_in}}}$



Xavier (uniform)

Uniform distribution

He (uniform)

$[-\text{lim}, \text{lim}] \rightarrow$

$$\text{lim} = \sqrt{\frac{6}{(\text{fan_in} + \text{fan_out})}}$$

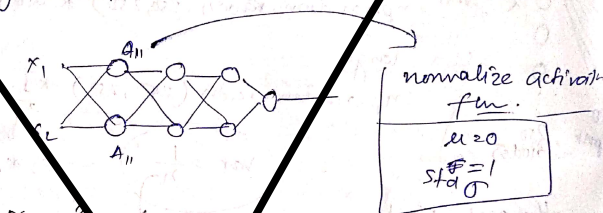
$[-\text{lim}, \text{lim}]$

$$\text{lim} = \sqrt{\frac{6}{(\text{fan_in})}}$$

What is Batch Norm?

Batch Normalization (BN) is an algorithmic method which makes the training of DNN faster & more stable.

It consists of normalizing activation vectors from hidden layers using the mean and variance of the current batch. This normalization step is applied right before (or right after) the nonlinear function.



Why use Batch Norm?

